Backend Integration

KSA SME Valuation Model — Backend Integration Reference

> **Purpose**
>
> This document is the single source of truth for backend developers integrating with the KSA SME Valuation model. It covers the full request schema, feature validation rules, auto-computed fields, inference pipeline order, and expected output format.

Quick Links

- Request Schema
- Feature Validation Rules
- Categorical Allowed Values
- Auto-Computed Features
- Optional Features & Defaults
- Cross-Field Validation Rules
- Inference Pipeline Order
- Feature Importance Rankings
- Output Format
- Currency Configuration
- Error Response Format
- Example Request & Response

Model Overview

| Property | Value |

|----------|-------|

| **Model file** | `models/ksa_valuation_model.onnx` |

Model type	Random Forest Regressor
Target	`valuation_sar`
R^2 Score	0.9436
MAE	2,162,941 SAR
RMSE	4,328,460 SAR
MAPE	14.78%
Training rows	12,000
Primary region	Saudi Arabia

Request Schema

The backend sends a JSON payload to the inference pipeline. The full field breakdown:

Field	Type	Required	Notes
`country`	string	✅ Yes	Default: `"Saudi Arabia"`
`industry`	string	✅ Yes	Must match allowed list
`city`	string	✅ Yes	Must match country's city list
`annual_revenue_sar`	float	✅ Yes	Min: 100,000
`annual_expenses_sar`	float	✅ Yes	Min: 0, must be ≤ revenue
`annual_profit_sar`	float	✅ Yes	Can be negative
`profit_margin`	float	✅ Yes	Range: −0.50 → 0.60
`number_of_employees`	int	✅ Yes	Min: 1
`years_in_operation`	float	✅ Yes	Min: 0
`monthly_customers`	int	✅ Yes	Min: 10
`revenue_growth_rate`	float	✅ Yes	Range: −0.50 → 1.00

| `competition_level` | int | ✅ Yes | Range: 1 → 10 |

| `owner_dependency_score` | int | ✅ Yes | Range: 1 → 10 |

| `scalability_score` | int | ✅ Yes | Range: 1 → 10 |

| `customer_growth_rate` | float | ◯ Optional | Default: `0.0` |

| `location_score` | int | ◯ Optional | Default: `5` |

| `revenue_stability_score` | int | ◯ Optional | Default: `6` |

| `market_demand_score` | int | 🤖 Auto | Computed from industry + city + country |

♨ **Do NOT send** `market_demand_score`

> This field is **auto-computed** by the server from `(industry, city, country)`. If you send it, it will be **ignored and overwritten**.

Feature Validation Rules

Financial Features

`annual_revenue_sar`

- **Type:** `float`
- **Minimum:** `100,000`
- **Maximum:** none
- **Required:** yes

`annual_expenses_sar`

- **Type:** `float`
- **Minimum:** `0`
- **Maximum:** must not exceed `annual_revenue_sar`
- **Required:** yes

`annual_profit_sar`

- **Type:** `float`
- **Minimum:** none (can be negative)

- **Maximum:** none
- **Required:** yes

`profit_margin`

- **Type:** `float` (decimal ratio, not percentage)
- **Minimum:** `-0.50`
- **Maximum:** `0.60`
- **Required:** yes
- **Example:** `0.30` = 30% margin

Business Size Features

`number_of_employees`

- **Type:** `int`
- **Minimum:** `1`
- **Maximum:** none
- **Required:** yes

`years_in_operation`

- **Type:** `float`
- **Minimum:** `0`
- **Maximum:** none
- **Required:** yes

`monthly_customers`

- **Type:** `int`
- **Minimum:** `10`
- **Maximum:** none
- **Required:** yes

Growth Features

`revenue_growth_rate`

- **Type:** `float` (decimal ratio)
- **Minimum:** `-0.50`
- **Maximum:** `1.00`
- **Required:** yes
- **Example:** `0.25` = 25% growth

`customer_growth_rate`

- **Type:** `float` (decimal ratio)
- **Minimum:** `-0.50`
- **Maximum:** `1.00`
- **Required:** ⚪ optional
- **Default:** `0.0`

Score Features (all 1–10 int)

Feature	Required	Default	Notes
`competition_level`	✅ Yes	—	1 = very low competition, 10 = very high
`owner_dependency_score`	✅ Yes	—	1 = fully team-run, 10 = owner-critical
`scalability_score`	✅ Yes	—	1 = not scalable, 10 = highly scalable
`location_score`	⚪ Optional	`5`	1 = bad location, 10 = prime location
`revenue_stability_score`	⚪ Optional	`6`	1 = volatile, 10 = very stable
`market_demand_score`	🤖 Auto	computed	Never input by user

Categorical Allowed Values

`industry`

Must be **exactly** one of:

```
Technology
```

```
Healthcare

Retail

Manufacturing

Education

Hospitality

Construction

Logistics

Finance

Food & Beverage

Real Estate

Professional Services
```

`country`

Must be **exactly** one of:

```
Saudi Arabia

United Arab Emirates

Qatar

Kuwait

Bahrain

Oman
```

> ✎ **KSA-First**
>
> Currently only `Saudi Arabia` has city-level validation enforced. Other countries are accepted for future expansion but do not have city-level restrictions yet.

`city` (Saudi Arabia)

When `country = "Saudi Arabia"`, city must be one of:

```
Riyadh

Jeddah

Dammam

Mecca

Medina

Khobar

Tabuk

Abha

Hail

Najran
```

Auto-Computed Features

`market_demand_score`

Computed server-side via `market_estimator.py` using:

```
market_demand_score = industry_base + city_adjustment
```

Clamped to range `[1, 10]`.

Industry Base Scores

Industry	Base Score
Technology	9
Healthcare	9
Finance	8
Manufacturing	8
Education	8
Construction	8
Logistics	8
Professional Services	8
Retail	7
Hospitality	7
Food & Beverage	7
Real Estate	7

KSA City Adjustments

City	Adjustment
Riyadh	+1
Jeddah	+1
Dammam	+1
Mecca	0
Medina	0

Khobar	0
Tabuk	−1
Abha	−1
Hail	−1
Najran	−1

Examples:

- Technology + Riyadh → `9 + 1 = 10`
- Retail + Abha → `7 + (-1) = 6`
- Healthcare + Jeddah → `9 + 1 = 10` (clamped)

Optional Features & Defaults

If the following fields are **missing or null** in the request, the server applies defaults automatically and logs the event.

Feature	Default Applied	Log file
`location_score`	`5`	`data/missing_feature_logs.csv`
`customer_growth_rate`	`0.0`	`data/missing_feature_logs.csv`
`revenue_stability_score`	`6`	`data/missing_feature_logs.csv`

Send optional fields if available

> Even though these have defaults, providing actual values will improve prediction accuracy since they still contribute to the model (see feature importance).

Cross-Field Validation Rules

These are validated **after** individual field validation passes.

Rule 1 — Expenses ≤ Revenue

```
annual_expenses_sar <= annual_revenue_sar
```

Rule 2 — Profit Consistency

```
| annual_profit_sar - (annual_revenue_sar - annual_expenses_sar) |
────────────────────────────────────────────────────────────────
 <= 0.05

        | annual_revenue_sar - annual_expenses_sar |
```

Tolerance: **5%**. The provided profit must be within 5% of computed `revenue - expenses`.

Rule 3 — Profit Margin Consistency

```
| profit_margin - (annual_profit_sar / annual_revenue_sar) | <=
0.05
```

Tolerance: **5 percentage points**.

Rule 4 — Revenue Per Customer Sanity

```
avg_revenue_per_customer = annual_revenue_sar / (monthly_customers
× 12)
```

Valid range: **5 SAR → 100,000 SAR per customer per year**.

Values outside this range will be rejected.

Inference Pipeline Order

The server executes these steps in sequence on every request:

```
1. Fill optional defaults
      ↓
2. Validate all inputs (individual + cross-field)
      ↓
3. Auto-estimate market_demand_score
      ↓
4. Build ONNX input tensors
      ↓
5. Run ONNX inference → valuation_sar
      ↓
6. Convert SAR → USD
      ↓
7. Return JSON response
```

⚠ **Validation before inference**

> If any validation rule fails, the request is rejected with a 400-level error **before** any model inference runs. This is intentional.

Feature Importance Rankings

Extracted from the trained Random Forest model. These values explain what the model weighs most when computing valuation.

Rank	Feature	Importance	Impact
1	`annual_revenue_sar`	**87.99%**	🔴 Dominant
2	`revenue_growth_rate`	1.92%	🟠 High
3	`scalability_score`	1.79%	🟠 High
4	`annual_expenses_sar`	1.62%	🟡 Medium
5	`annual_profit_sar`	1.43%	🟡 Medium
6	`customer_growth_rate`	1.01%	🟡 Medium
7	`revenue_stability_score`	0.76%	🟢 Low
8	`owner_dependency_score`	0.63%	🟢 Low
9	`monthly_customers`	0.47%	🟢 Low
10	`profit_margin`	0.46%	🟢 Low
11	`number_of_employees`	0.35%	🟢 Low
12	`market_demand_score`	0.33%	🟢 Low
13	`location_score`	0.29%	🟢 Low
14	`competition_level`	0.29%	🟢 Low
15	`years_in_operation`	0.27%	🟢 Low

Insight

> `annual_revenue_sar` dominates at ~88% importance. This means the model is fundamentally a **revenue-multiple estimator** adjusted by growth potential and operational quality signals.

Output Format

Every successful prediction returns:

```json
{
  "valuation_sar": 19882174.0,
  "valuation_usd": 5298599.37,
  "market_demand_score_used": 10
}
```

Field	Type	Description
`valuation_sar`	float	Estimated business valuation in Saudi Riyal
`valuation_usd`	float	Same valuation converted to US Dollars
`market_demand_score_used`	int	The auto-computed score (1–10) for transparency

Currency Configuration

Parameter	Value
Base currency	SAR (Saudi Riyal)
Secondary currency	USD
SAR → USD rate	`0.2665`
USD → SAR rate	`3.75`
Config file	`config/currency_config.json`

Conversion formula:

```
valuation_usd = valuation_sar × 0.2665
```

⚠ **Exchange rate updates**

> The rate is hardcoded in `src/currency_converter.py` and mirrored in `config/currency_config.json`. To update it, call `update_exchange_rate(new_rate)` and update the JSON file. The model itself does **not** need to be retrained for rate changes.

Error Response Format

When validation fails, the API returns an error with **all validation errors collected at once** (not just the first one):


```
{
  "error": "Input validation failed with 2 error(s):",
  "details": [
    "annual_revenue_sar must be >= 100,000 SAR. Got: 50000",
    "profit_margin must be between -0.50 and 0.60. Got: 0.75"
  ]
}
```

Feature Order (ONNX Tensor Input Order)

For direct ONNX consumers

> If you are calling the ONNX model directly (not through `inference.py`), you must supply features in this **exact order**. Categorical features come **before** numerical features.

```
Categorical (StringTensorType [None, 1]):

  1. industry

  2. city

  3. country



Numerical (FloatTensorType [None, 1]):

  4.  years_in_operation

  5.  number_of_employees

  6.  annual_revenue_sar

  7.  annual_expenses_sar

  8.  annual_profit_sar

  9.  profit_margin

  10. revenue_growth_rate

  11. customer_growth_rate

  12. monthly_customers

  13. market_demand_score      ← auto-computed, must be filled
before ONNX call

  14. competition_level

  15. owner_dependency_score

  16. scalability_score
```

```
17. location_score

18. revenue_stability_score
```

Example Request & Response

Minimal Request (with optional fields omitted)


```json
{
  "country": "Saudi Arabia",
  "industry": "Technology",
  "city": "Riyadh",
  "annual_revenue_sar": 8500000,
  "annual_expenses_sar": 5950000,
  "annual_profit_sar": 2550000,
  "profit_margin": 0.30,
  "number_of_employees": 55,
  "years_in_operation": 7,
  "monthly_customers": 180,
  "revenue_growth_rate": 0.25,
  "competition_level": 6,
  "owner_dependency_score": 4,
  "scalability_score": 8
```

```
}
```

Server-Side Processing Log

```
[INFO] Missing optional feature: location_score -> default 5 applied

[INFO] Missing optional feature: customer_growth_rate -> default 0.0 applied

[INFO] Missing optional feature: revenue_stability_score -> default 6 applied

[INFO] Auto market_demand_score = 10 (Technology in Riyadh)
```

Response


```json
{

  "valuation_sar": 19882174.0,

  "valuation_usd": 5298599.37,

  "market_demand_score_used": 10

}
```

File Reference Map

| File | Purpose |

|------|---------|

| `src/config.py` | Central config: feature lists, paths, allowed values |

| `src/validation.py` | All input validation logic |

`src/market_estimator.py`	Auto market demand estimation
`src/currency_converter.py`	SAR ↔ USD conversion
`src/inference.py`	Full prediction pipeline
`src/preprocess.py`	Data preprocessing + ColumnTransformer
`src/train.py`	Model training + config JSON export
`src/export_onnx.py`	ONNX export
`config/feature_order.json`	Machine-readable feature order
`config/feature_importance.json`	Feature importance scores
`config/validation_rules.json`	Machine-readable validation spec
`config/region_config.json`	Region + city configuration
`config/currency_config.json`	Exchange rate config
`data/missing_feature_logs.csv`	Runtime log of optional field defaults
`models/ksa_valuation_model.onnx`	Production ONNX model

Generated: 2026-04-04 | Model version: 1.0.0 | Region: KSA-first